

SIH 2026 — PS 26127

FULL API CONTRACT & TEAM GUIDE

City-Wide AI Engine for Multi-Camera ANPR Trajectory Tracking

Purpose: Source-of-truth guide for a 4-person feature-based team. It defines endpoint ownership, request/response contracts, data flow and integration rules.

MVP rule: Keep the contract stable during the 10-day build. Add endpoints only when a demonstrated feature needs them.

1. SYSTEM ARCHITECTURE

Camera/Video → AI Pipeline → Event Contract → Backend → PostgreSQL/PostGIS/Redis → REST/WebSocket → Frontend/GIS

Ownership: Person 1 = Camera + Detection + Tracking. Person 2 = ANPR + Re-ID + Global Identity. Person 3 = Backend + Database + Trajectory. Person 4 = Frontend + GIS + Traffic Visualization.

Boundary: AI does not directly write to the database. AI produces events; Backend owns persistence, identity resolution, trajectory and business logic; Frontend consumes Backend APIs.

2. COMMON RULES

- Base path: /api/v1
- JSON is the default format.
- Timestamps use ISO-8601 UTC.
- IDs are strings; confidence values are 0.0–1.0.
- Bounding boxes use x1,y1,x2,y2 pixel coordinates.
- Embeddings are float arrays and include model/version metadata.
- HTTP: 200/201 success; 400 validation; 401 unauthenticated; 403 forbidden; 404 not found; 409 conflict; 422 schema validation; 500 server error.
- Never expose raw embeddings or sensitive individual-vehicle data to public users.

3. SHARED DETECTION EVENT

This is the key contract between the AI/identity features and Backend.

```
{
  "event_id": "evt_01H...",
  "camera_id": "CAM_001",
  "observed_at": "2026-08-31T10:30:20Z",
  "local_track_id": "CAM_001_T_73",
  "vehicle_type": "car",
  "plate_number": "WB12AB1234",
  "plate_confidence": 0.94,
  "vehicle_embedding": [0.012, -0.084, 0.221],
  "embedding_model": "reid-model-v1",
  "embedding_version": "1.0",
  "vehicle_confidence": 0.91,
  "bounding_box": {"x1": 412, "y1": 220, "x2": 690, "y2": 530},
  "frame_reference": "frames/CAM_001/evt_01H.jpg"
}
```

Important: Do not send embeddings for every frame. Send a representative/best observation for a tracked vehicle, or use the batch endpoint.

4. AI / CAMERA FEATURE — ENDPOINTS

Person 1: camera ingestion, OpenCV, YOLO, tracking and event generation. **Person 2** supplies ANPR/Re-ID fields to the same event.

4.1 Camera heartbeat

POST /api/v1/cameras/{camera_id}/heartbeat

Owner/flow: AI → Backend

What it does: Reports that the camera/AI worker is alive.

Request:

```
{"status": "online", "fps": 24, "processing_latency_ms": 42}
```

Response:

```
{"camera_id": "CAM_001", "status": "online", "last_seen": "2026-08-31T10:30:20Z"}
```

Notes: Send every 15–30 seconds. Backend can mark a camera offline when heartbeats stop.

4.2 Single detection event

POST /api/v1/events/detection

Owner/flow: AI → Backend

What it does: Sends one representative vehicle observation.

Request:

```
{"camera_id": "CAM_001", "observed_at": "2026-08-31T10:30:20Z", "local_track_id": "T_73", "vehicle_type": "car", "plate_number": "WB12AB1234", "plate_confidence": 0.94, "vehicle_embedding": [0.012, -0.084, 0.221], "embedding_model": "reid-model-v1", "embedding_version": "1.0", "vehicle_confidence": 0.91, "bounding_box": {"x1": 412, "y1": 220, "x2": 690, "y2": 530}}
```

Response:

```
{"event_id": "evt_01H...", "status": "accepted", "global_vehicle_id": "V1024"}
```

Notes: Use for the simplest MVP flow.

4.3 Batch detection events

POST /api/v1/events/batch

Owner/flow: AI → Backend

What it does: Sends multiple detection events in one HTTP request.

Request:

```
{"events": [{"camera_id": "CAM_001", "observed_at": "2026-08-31T10:30:20Z", "local_track_id": "T_73", "vehicle_type": "car", "plate_number": "WB12AB1234", "plate_confidence": 0.94, "vehicle_embedding": [0.012, -0.084, 0.221], "embedding_model": "reid-model-v1", "embedding_version": "1.0", "vehicle_confidence": 0.91, "bounding_box": {"x1": 412, "y1": 220, "x2": 690, "y2": 530}}]}
```

Response:

```
{"accepted": 1, "rejected": 0, "event_ids": ["evt_01H..."]}
```

Notes: Useful for scale; optional until single-event flow works.

4.4 Camera status

GET /api/v1/cameras/{camera_id}/status

Owner/flow: Frontend/Backend → Backend

What it does: Reads current camera/AI health.

Response:

```
{"camera_id": "CAM_001", "status": "online", "last_seen": "2026-08-31T10:30:20Z", "fps": 24, "processing_latency_ms": 42}
```

Notes: Heartbeat updates the values returned here.

5. IDENTITY / ANPR / RE-ID FEATURE

Person 2: ANPR/OCR, plate quality, embeddings, Re-ID and cross-camera identity resolution.

5.1 Vehicle search

GET /api/v1/vehicles/search?plate_number=WB12AB1234

Owner/flow: Frontend → Backend

What it does: Searches authorized vehicle records.

Response:

```
{ "items": [ { "vehicle_id": "V1024", "plate_number": "WB12AB1234", "last_seen": "2026-08-31T10:30:20Z" } ], "total": 1 }
```

Notes: Support time-range/type filters if needed.

5.2 Vehicle details

GET /api/v1/vehicles/{vehicle_id}

Owner/flow: Frontend → Backend

What it does: Returns the authorized summary of one global vehicle.

Response:

```
{ "vehicle_id": "V1024", "plate_number": "WB12AB1234", "vehicle_type": "car", "first_seen": "2026-08-31T09:52:10Z", "last_seen": "2026-08-31T10:30:20Z", "match_confidence": 0.89 }
```

Notes: Do not return raw embeddings.

5.3 Vehicle events

GET /api/v1/vehicles/{vehicle_id}/events

Owner/flow: Frontend → Backend

What it does: Returns camera observations associated with the global vehicle.

Response:

```
{ "vehicle_id": "V1024", "events": [ { "event_id": "evt_01", "camera_id": "CAM_001", "observed_at": "2026-08-31T10:02:31Z", "plate_confidence": 0.94, "match_confidence": 0.89 }, { "event_id": "evt_02", "camera_id": "CAM_002", "observed_at": "2026-08-31T10:08:14Z", "plate_confidence": 0.91, "match_confidence": 0.86 } ] }
```

Notes: Events are raw observations; trajectory is the reconstructed journey.

5.4 Vehicle trajectory

GET /api/v1/vehicles/{vehicle_id}/trajectory

Owner/flow: Frontend → Backend

What it does: Returns the reconstructed chronological path.

Response:

```
{ "vehicle_id": "V1024", "points": [ { "camera_id": "CAM_001", "timestamp": "2026-08-31T10:02:31Z", "lat": 22.5726, "lon": 88.3639 }, { "camera_id": "CAM_002", "timestamp": "2026-08-31T10:08:14Z", "lat": 22.5750, "lon": 88.3700 } ] }
```

Notes: Backend reconstructs the path; frontend visualizes it.

6. BACKEND / DATABASE / TRAJECTORY FEATURE

Person 3: FastAPI, PostgreSQL/PostGIS, persistence, global identity, trajectory, camera management, analytics, authentication and WebSockets.

6.1 List cameras

GET /api/v1/cameras

Owner/flow: Frontend → Backend

What it does: Returns cameras the caller may see.

Response:

```
{ "items": [ { "camera_id": "CAM_001", "name": "Junction  
A", "lat": 22.5726, "lon": 88.3639, "status": "online" } ], "total": 1 }
```

6.2 Camera details

GET /api/v1/cameras/{camera_id}

Owner/flow: Frontend → Backend

What it does: Returns metadata for one camera.

Response:

```
{ "camera_id": "CAM_001", "name": "Junction  
A", "lat": 22.5726, "lon": 88.3639, "status": "online", "last_seen": "2026-08-31T10:30:20Z" }
```

Notes: Never expose RTSP URLs or infrastructure secrets.

6.3 Traffic overview

GET /api/v1/traffic

Owner/flow: Frontend → Backend

What it does: Returns current area-level traffic metrics.

Response:

```
{ "timestamp": "2026-08-31T10:30:20Z", "vehicle_count": 1240, "average_speed_kmh": 18, "congestion_level": "high" }
```

Notes: Document the aggregation window used.

6.4 Camera traffic

GET /api/v1/traffic/{camera_id}

Owner/flow: Frontend → Backend

What it does: Returns traffic metrics for one camera/zone.

Response:

```
{ "camera_id": "CAM_001", "timestamp": "2026-08-31T10:30:20Z", "vehicle_count": 71, "average_speed_kmh": 18, "density":  
0.78, "congestion_level": "high" }
```

6.5 Route traffic

GET /api/v1/traffic/route/{route_id}

Owner/flow: Frontend → Backend

What it does: Returns aggregated traffic for a logical road route.

Response:

```
{ "route_id": "R_101", "timestamp": "2026-08-31T10:30:20Z", "vehicle_count": 320, "average_speed_kmh": 22, "congestion_  
level": "moderate" }
```

6.6 Analytics overview

GET /api/v1/analytics/overview

Owner/flow: Frontend → Backend

What it does: Returns dashboard KPIs.

Response:

```
{ "active_cameras": 48, "vehicles_seen_last_5m": 1240, "high_congestion_zones": 7, "events_last_5m": 5420 }
```

6.7 Login

POST /api/v1/auth/login

Owner/flow: Frontend → Backend

What it does: Authenticates a user.

Request:

```
{ "username": "operator01", "password": "*****" }
```

Response:

```
{"access_token":"","token_type":"bearer","expires_in":3600,"user":{"id":"U01","role":"authorized_operator"}}
```

Notes: Use HTTPS and secure password storage; never log credentials.

6.8 Current user

GET /api/v1/auth/me

Owner/flow: Frontend → Backend

What it does: Returns user identity and permissions.

Response:

```
{"user_id":"U01","role":"authorized_operator","permissions":["vehicle:read","trajectory:read"]}
```

Notes: Backend enforces authorization.

6.9 Refresh token

POST /api/v1/auth/refresh

Owner/flow: Frontend → Backend

What it does: Obtains a new access token if refresh-token auth is used.

Request:

```
{"refresh_token":""}
```

Response:

```
{"access_token":"","token_type":"bearer","expires_in":3600}
```

Notes: Optional for MVP.

7. REAL-TIME WEBSOCKET FEATURE

Use WebSockets for continuously changing data. Use REST to load snapshots/history.

7.1 Public traffic stream

WS /api/v1/ws/traffic

Owner/flow: Backend → Public Frontend

What it does: Continuously pushes anonymous traffic updates.

Response:

```
{"type": "traffic_update", "timestamp": "2026-08-31T10:30:20Z", "camera_id": "CAM_001", "vehicle_count": 71, "average_speed_kmh": 18, "congestion_level": "high"}
```

Notes: Never send plate numbers or private vehicle identities.

7.2 Authorized vehicle stream

WS /api/v1/ws/vehicle/{vehicle_id}

Owner/flow: Backend → Authorized Frontend

What it does: Pushes new observations/trajectory updates for a monitored vehicle.

Response:

```
{"type": "vehicle_update", "vehicle_id": "V1024", "camera_id": "CAM_002", "timestamp": "2026-08-31T10:30:20Z", "match_confidence": 0.89, "lat": 22.5750, "lon": 88.3700}
```

Notes: Require authorization.

7.3 Optional alert stream

WS /api/v1/ws/alerts

Owner/flow: Backend → Authorized Frontend

What it does: Pushes suspicious-route/security alerts.

Response:

```
{"type": "alert", "alert_id": "A102", "vehicle_id": "V1024", "severity": "high", "reason": "repeated_zone_visit", "timestamp": "2026-08-31T10:30:20Z"}
```

Notes: Optional; only build if the demo includes alerts.

8. ERROR CONTRACT

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid camera_id",
    "details": [{"field": "camera_id", "reason": "required"}],
    "request_id": "req_01H..."
  }
}
```

Recommended codes: VALIDATION_ERROR, UNAUTHORIZED, FORBIDDEN, NOT_FOUND, CONFLICT, RATE_LIMITED, INTERNAL_ERROR. Keep a request/correlation ID in backend logs.

9. AUTHORIZATION

Role	Access
Public	Anonymous traffic, congestion and non-sensitive route/camera information
Authorized Operator	Vehicle search, individual events, trajectory and authorized camera details
Admin	User/camera management plus operator capabilities

Backend performs the real authorization check. Hiding a button in React is not security.

10. DATABASE ENTITIES

- cameras: camera_id, name, latitude, longitude, status, last_seen
- vehicles: global vehicle_id, normalized plate, vehicle_type, first_seen, last_seen
- detections/events: event_id, camera_id, local_track_id, timestamp, plate data, embedding reference, bounding box, confidence
- trajectories: vehicle_id, ordered points/segments, timestamps, camera IDs and route information
- traffic_metrics: camera/route ID, time window, vehicle count, speed, density, congestion
- users: user_id, role and authentication metadata
- alerts: alert_id, vehicle/zone, type, severity, timestamp, status

11. END-TO-END EXAMPLE

1. Camera A video enters Person 1's AI pipeline.
2. YOLO detects a car; tracker assigns local_track_id T_73.
3. Person 2's ANPR reads the plate and Re-ID produces an embedding.
4. AI sends POST /api/v1/events/detection.
5. Backend validates and stores the event.
6. Backend compares plate/embedding/time/location with existing identities.
7. Backend assigns or updates global_vehicle_id V1024.
8. Backend updates the trajectory.
9. Authorized frontend calls GET /vehicles/V1024/trajectory and draws it on MapLibre.
10. While monitoring V1024, frontend subscribes to WS /ws/vehicle/V1024.
11. Traffic metrics are published through WS /ws/traffic for the public dashboard.

12. 4-PERSON OWNERSHIP

Person	Feature	Contract
1	Camera, OpenCV, YOLO, tracking, local IDs	Heartbeat + detection event
2	ANPR/OCR, embeddings, Re-ID, global identity	Detection identity fields + matching
3	FastAPI, DB, PostGIS, trajectory, auth, WebSockets	Backend endpoints
4	React, GIS, dashboards, real-time visualization	Consumes REST + WebSockets

13. 10-DAY API IMPLEMENTATION ORDER

- Day 1: Freeze event schema, IDs, timestamps, authentication and error format.
- Day 2: Backend skeleton + DB tables + event receiver + frontend API client.
- Day 3: Detection/tracking produces valid events; Backend stores them.
- Day 4: Integrate ANPR/OCR and embedding fields.
- Day 5: Global vehicle matching and vehicle APIs.
- Day 6: Trajectory API + MapLibre visualization.
- Day 7: Traffic REST + traffic WebSocket.
- Day 8: End-to-end integration; avoid contract changes.
- Day 9: Validation, security, error handling and edge-case tests.

- Day 10: Freeze API, document Swagger/OpenAPI and rehearse demo.

14. AI-AGENT COLLABORATION RULES

- Tell every coding agent which feature it owns and which contract it must follow.
- Agents may refactor internal files, but must not silently rename/remove API fields or endpoints.
- Contract changes require a PR/change request and an updated shared contract.
- Use mocks/stubs generated from the contract so people can work in parallel.
- Keep one source of truth: docs/API_CONTRACT.md or an OpenAPI specification.
- Run contract/integration tests before merging.

15. MVP ENDPOINT CHECKLIST

- POST /api/v1/cameras/{camera_id}/heartbeat
- POST /api/v1/events/detection
- POST /api/v1/events/batch
- GET /api/v1/cameras
- GET /api/v1/cameras/{camera_id}
- GET /api/v1/cameras/{camera_id}/status
- GET /api/v1/vehicles/search
- GET /api/v1/vehicles/{vehicle_id}
- GET /api/v1/vehicles/{vehicle_id}/events
- GET /api/v1/vehicles/{vehicle_id}/trajectory
- GET /api/v1/traffic
- GET /api/v1/traffic/{camera_id}
- GET /api/v1/traffic/route/{route_id}
- GET /api/v1/analytics/overview
- POST /api/v1/auth/login
- GET /api/v1/auth/me
- WS /api/v1/ws/traffic
- WS /api/v1/ws/vehicle/{vehicle_id}
- Optional: POST /api/v1/auth/refresh
- Optional: WS /api/v1/ws/alerts

FINAL PRINCIPLE

AI produces observations. Backend turns observations into intelligence. Frontend turns intelligence into decisions.
Keep this boundary clear so all four people can work in parallel without waiting for one another.